IU2541230787 - CPP74 - PROJECT REPORT

Employee Leave Management

Enrollment No.: IU2541230787

Student Name: Krish Singh

Semester: 3

Subject: C++ Guided Project

1. Introduction

Employee Leave Management is a console-based, object-oriented C++17 application that replaces paper or e-mail leave requests. Employees apply for leave, managers approve or reject, and the system automatically counts working days, checks leave balances and enforces company rules. All data is stored in files and restored when the program is started again.

2. Problem Statement

Manual leave handling leads to miscounted days (weekends and holidays), double-booked dates, overdrawn leave balances, too many people absent from one team, and no record of who approved what. The project provides a single program that validates every request against clear rules and keeps a permanent, auditable record.

3. Objectives

- Model employees, managers, leave requests, approvals and holidays as cooperating classes.
- Demonstrate classes, encapsulation, constructors/destructors, inheritance, runtime polymorphism, dynamic memory, file handling, validation and exception handling.
- Enforce realistic rules: yearly quotas, notice periods, back-dating, overlap detection, department cover limit.
- Provide Add / Display / Search / Update / Delete, the leave process, reports and Save / Load through the required 9-option menu.
- Prove correctness with automated tests (41 end-to-end, 84 unit checks, 9 compile-fail proofs) and memory-sanitizer runs.

4. Functional Requirements

- FR1 Add employees, managers, public holidays and leave requests with full input validation (name, e-mail, real calendar dates).
- FR2 Display each record type as a table, and 'Everything' through one polymorphic loop.
- FR3 Search employees/managers/holidays/requests by text, and requests overlapping a date range.
- FR4 Update employees, managers, holidays and Pending requests (days and balance recalculated).

- FR5 Delete employees, managers and holidays, protected by referential rules (no deletion with active leave or recorded decisions).
- FR6 Leave process: apply, approve/reject by an authorised manager, cancel, check balance.
- FR7 Reports: employee balance, department summary, who is on leave on a date, monthly calendar, overall summary, CSV export, audit log.
- FR8 Save, reload, backup, load demo data; automatic load at start and save prompt at exit.
- FR9 Business rules: Annual 18 days/yr (3 days notice), Sick 10 (back-date 7), Casual 7 (max 3 days), Parental 30 (14 days notice), Unpaid unlimited (7 days notice); no overlap; at most half of a department away per day; rejection needs a reason.

5. System / Project Flow

USER -> MAIN MENU (ConsoleUI) -> LeaveSystem (rules) -> Entity classes (Employee, Manager, LeaveRequest, Approval, Holiday, LeavePolicy) -> FileManager -> DATA FILES (Data/*.dat, audit.log)

Start: load existing records -> user performs operations (kept in memory, each change written to the audit log) -> Save (atomic write of five .dat files) -> Exit. main.cpp only creates the LeaveSystem and ConsoleUI objects and calls run().

6. Class Descriptions

Class	Data Members	Member Functions	Responsibility
Record (abstract)	id_	id(), kind(), display(), row(), serialize(), matches()	Common root of every stored entity; forces each entity to know how to show and save itself.
Person (abstract)	name_, department_	setName(), setDepartment(), role(), matches()	Shared, validated identity of Employee and Manager.
Employee	designation_, email_, joined_	Employee(), Employee(...), setEmail(), display(), serialize(), fromFields()	A staff member who can apply for leave.
Manager	title_, hr_	Manager(), authorises(Employee), role(), display()	Approves/rejects leave; HR managers may decide for every department.
LeaveRequest	employeeId_, type_, from_, to_, applied_, reason_, days_, status_	LeaveRequest(), calculateDays(), countWorkingDays(), overlaps(), covers(), setStatus(), display()	One leave application and its life-cycle (Pending, Approved, Rejected, Cancelled).
Approval	requestId_, managerId_, decision_, comment_, decidedOn_	Approval(), approve(), reject(), display()	A manager's recorded decision (audit trail) on one request.
Holiday	date_, name_	Holiday(), setDate(), setName(), display()	Public holiday - not counted as a leave day.
LeavePolicy + 5 subclasses	(none; rules are virtual functions)	code(), yearlyQuota(), maxConsecutive(), minNoticeDays(), maxBackdateDays(), paid(), check()	AnnualLeave, SickLeave, CasualLeave, ParentalLeave, UnpaidLeave each define their own limits (polymorphism).
LeaveSystem	fm_, audit_, 5 x PtrList, policies_ (LeavePolicy**), today_, dirty_	addX(), searchX(), updateX(), deleteX(), applyLeave(), decide(), cancelLeave(), balance(), reports, save(), load()	Controller: owns all data and enforces every business rule; never reads the keyboard.
ConsoleUI	sys_ (LeaveSystem&)	run(), addMenu() ... exitMenu(), applyForLeave(), reviewQueue()	Menus and prompts only; catches exceptions and prints friendly messages.
FileManager	dir_	save<T>(), readLines(), backup(), writeLines(), checksum()	Atomic file save/load with header, record count and checksum.
PtrList<T>	data_ (T**), size_, cap_	add(), operator[], removeAt(),	Owning dynamic array that

		clear(), ~PtrList()	doubles when full and deletes its objects.
AuditLog / Date	path_ / y_, m_, d_	log(), tail() / weekday(), addDays(), parse(), today()	Change history file; calendar arithmetic written from scratch.

7. OOP Concepts Used

- Classes & objects: 13 classes (5 entities, 6 policies, controller, UI, file manager, container, date, audit log).
- Encapsulation: all data members are private (Record::id_ has no setter); changes go through validating setters. Compile-fail test proves private access is rejected.
- Constructors: default and parameterized constructors on all entities; Date validates in its constructor.
- Destructors: virtual ~Record() and ~LeavePolicy(); ~PtrList() and ~LeaveSystem() release dynamic memory.
- Inheritance: Record -> Person -> Employee/Manager; Record -> LeaveRequest/Approval/Holiday; LeavePolicy -> 5 leave types (multi-level).
- Runtime polymorphism: Record* calls display()/row()/serialize(); LeavePolicy* calls quota/notice/limit functions that differ per leave type.
- Dynamic memory: PtrList<T> (new T*[], doubling, delete[]), LeavePolicy** array, objects created with new and owned by lists; unique_ptr guards against leaks when a rule throws. LeakSanitizer reports zero leaks.
- Exception handling: ElmError hierarchy (Validation, Rule, NotFound, File, Parse); failed operations leave data unchanged.
- Templates and STL: PtrList<T>, FileManager::save<T>, std::vector, std::map, std::sort, std::filesystem.

8. File Handling and Data Persistence

Five record files (employees.dat, managers.dat, requests.dat, approvals.dat, holidays.dat) plus audit.log live in Data/. Each .dat starts with a header '#ELM1 <kind> <count> <checksum>' followed by one tab-separated line per record (tabs/newlines escaped). Save writes <file>.tmp and renames it, so a crash never leaves a half-written file. Load parses every line; a corrupt line is skipped with a warning, a wrong header raises FileError, a missing file means first run, and record-count/checksum mismatches and dangling references are reported. ID counters are rebuilt from the loaded data. Test T40 closes and reopens the program and confirms the data is still present.

9. UML Diagrams

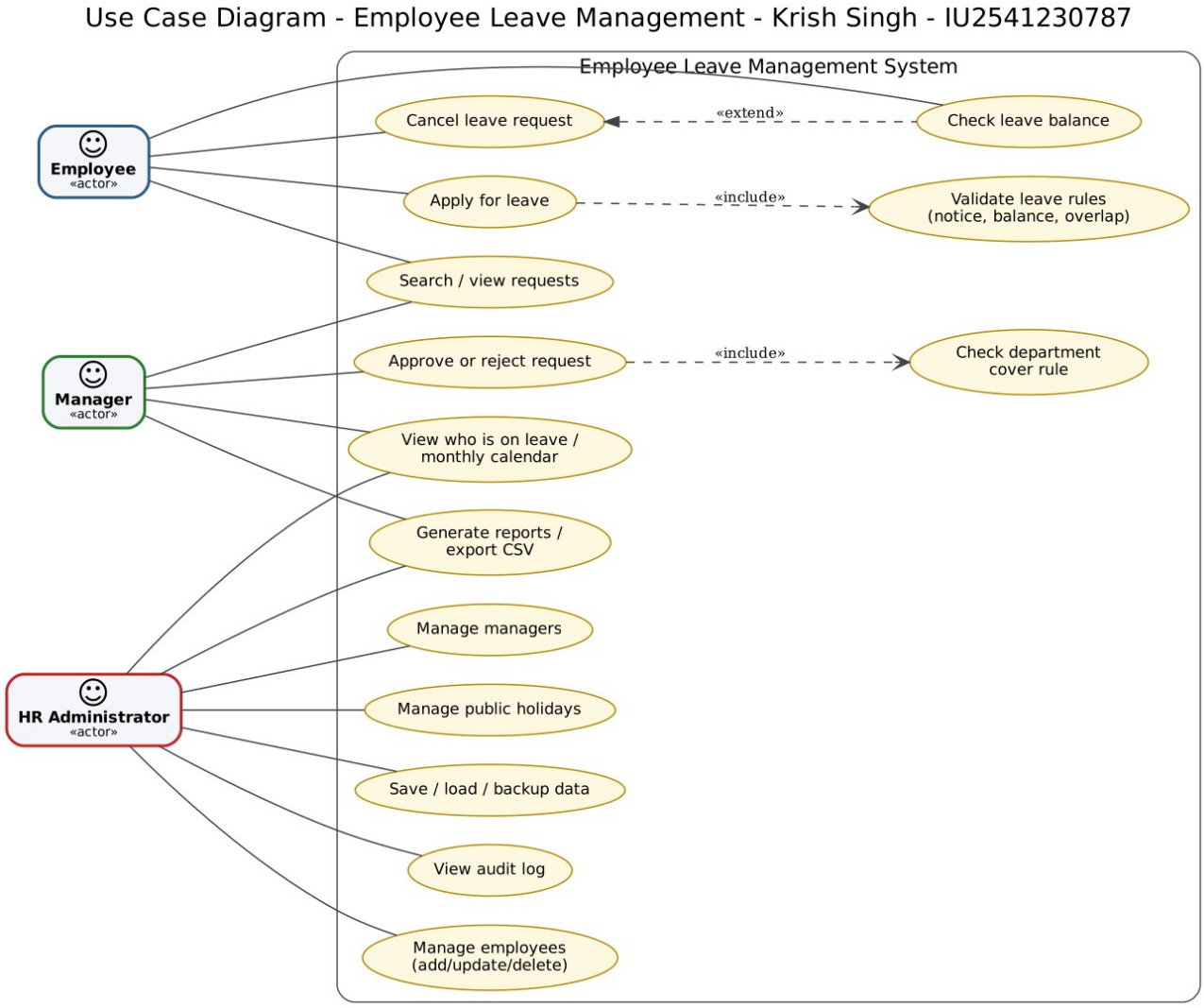


Figure 1 - Use Case Diagram (3 actors, 14 use cases)

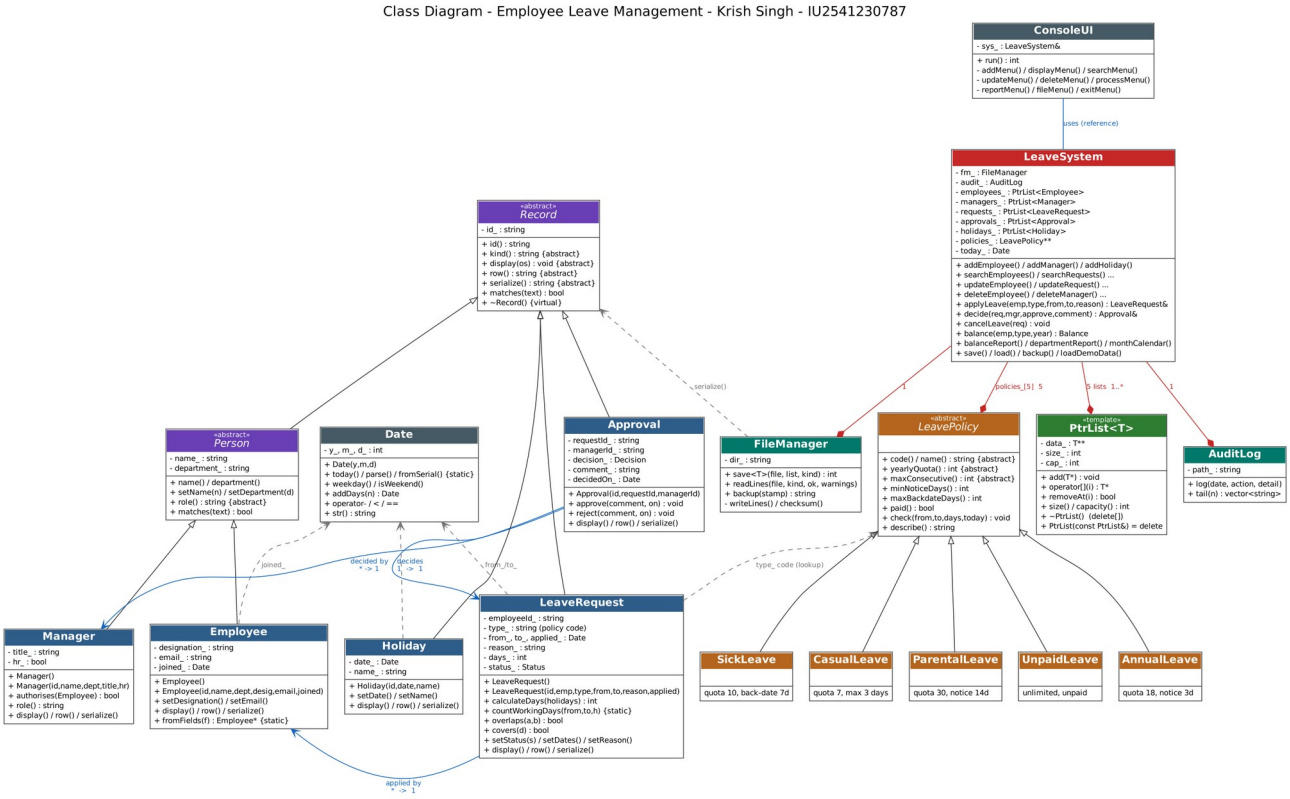


Figure 2 - Class Diagram

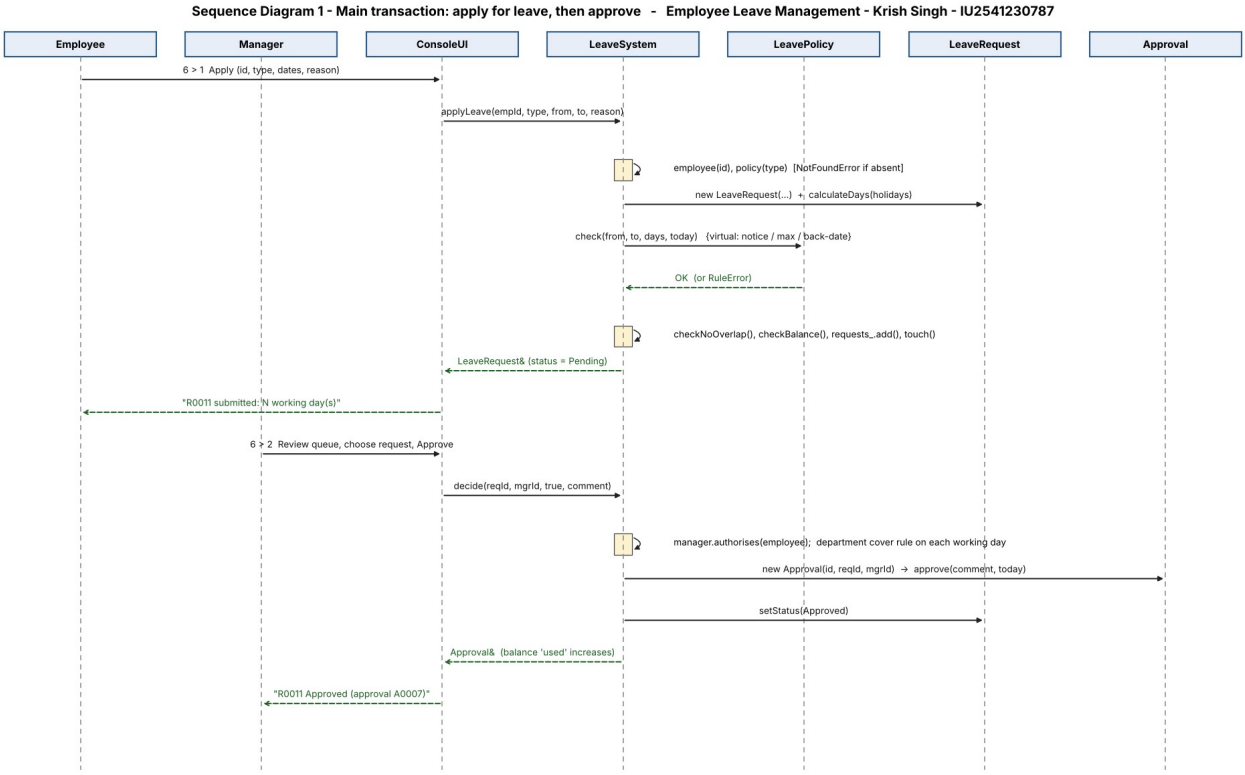


Figure 3 - Sequence Diagram 1: apply for leave and approve

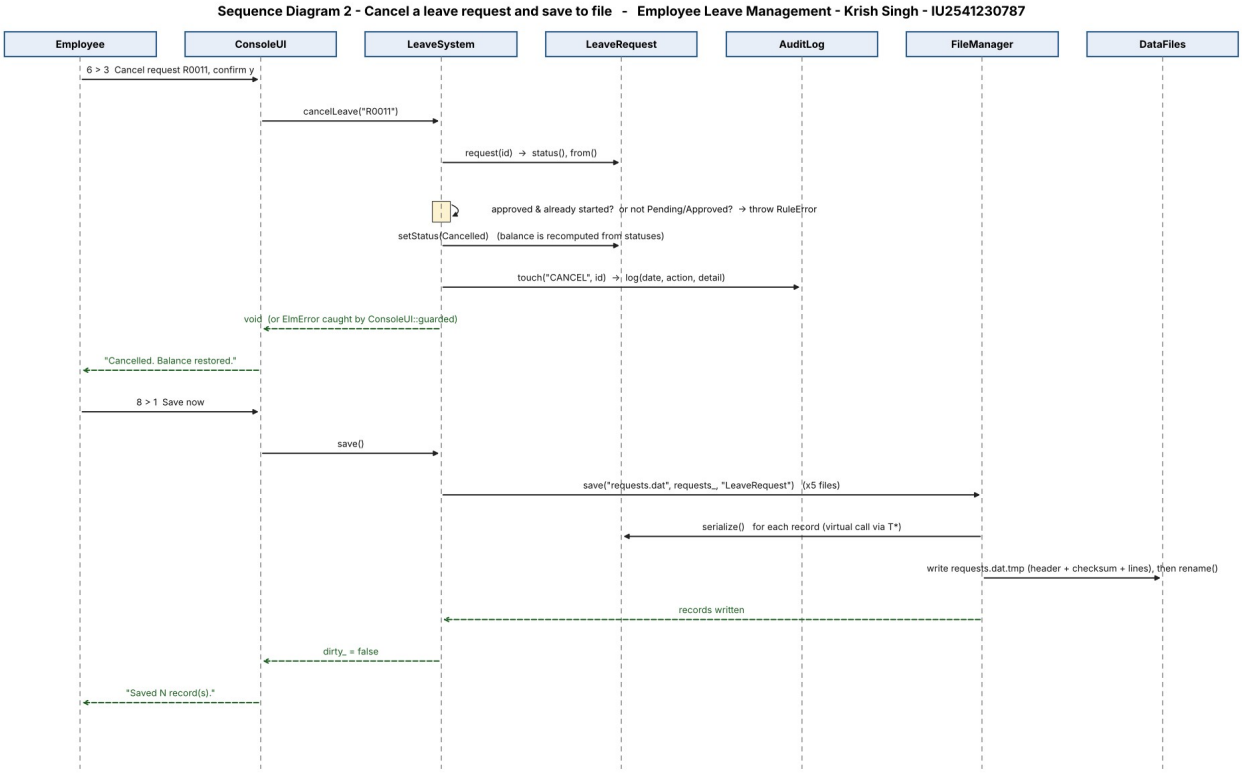


Figure 4 - Sequence Diagram 2: cancel a request and save

10. Implementation Details

- Source: 13 .cpp and 17 headers in src/ and include/elm/, about 2,300 lines including tests; compiled with -std=c++17 -Wall -Wextra -Wpedantic -Wold-style-cast -Wshadow -Werror (zero warnings).
- Working days: LeaveRequest::calculateDays walks the date range, skipping Saturday/Sunday and holidays; the result is stored so later holiday edits cannot change approved history.
- Date class: own implementation of civil-calendar day numbers; leap years, weekday and date arithmetic are unit-tested.
- Balance is derived, not stored: sum of Approved (used) and Pending days per type per year, so cancelling a request automatically restores the balance.
- Cover rule: on approval, every working day of the request is checked; no more than half of the employee's department (minimum 1) may be on approved leave.
- Safety: all multi-step changes validate first and modify data last; decide() builds the Approval before touching the request status.
- Testing tools: Tests/run_tests.py (stdin-driven end-to-end), Tests/unit_tests.cpp (ASan + UBSan), Tests/compile_fail_checks.sh.

11. Testing and Results

The 15 required test cases are listed below (suite ID in brackets). The complete suite of 41 end-to-end tests is in Tests/results.json with a screenshot per test in Tests/Screenshots/; in addition 84 unit checks (Tests/unit_tests.cpp) and 9 compile-fail proofs were run.

Test ID	Input / Operation	Expected	Actual	Status
T01	Add employee (valid) [T01] - Krish Singh / Engineering / Developer / krish@acme.com / 2025-08-01	Added employee E001	As expected	PASS
T02	Add employee: invalid e-mail [T02] - e-mail 'bad' then valid	Enter a valid e-mail ; Added employee E001	As expected	PASS
T03	Display all employees [T07] - Display > Employees	Aarav Sharma ; Neha Gupta ; 8 record(s)	As expected	PASS
T04	Search employee by department [T09] - text 'sales'	Vikram Rao ; Sana Khan ; Arjun Verma ; 3 record(s)	As expected	PASS
T05	Update employee [T12] - E001: new designation 'Senior Engineer', others kept	Updated E001 ; Senior Engineer	As expected	PASS
T06	Update pending leave request [T14] - R0005 -> 2026-11-09..2026-11-10	Updated R0005 - now 2 working day(s)	As expected	PASS
T07	Delete employee with pending leave [T16] - E003 has pending/approved leave	still has	As expected	PASS
T08	Apply for leave (valid) [T19] - E001 AL 2026-11-16..2026-11-17 'Visit home'	submitted: 2 working day(s) ; Pending	As expected	PASS
T09	Apply for leave: overlap [T20] - E001 AL 2026-10-30 (already on R0001)	overlap with Approved request R0001	As expected	PASS
T10	Apply: insufficient balance [T21] - 11 days, then 5 more with only 4 left	submitted: 11 working day(s) ; insufficient Annual Leave balance	As expected	PASS
T11	Manager approves request [T24] - R0003 by M001	R0003 Approved	As expected	PASS
T12	Manager rejects (reason mandatory) [T25] - R0006 by M001; empty reason first	cannot be empty ; R0006 Rejected	As expected	PASS
T13	Department cover rule [T27] - 3rd Engineering approval on same day blocked	R0011 Approved ; cover rule	As expected	PASS
T14	Cancel a pending request [T28] - R0005	Cancelled. Balance restored ; Cancelled	As expected	PASS
T15	Persistence across restarts [T40] - run 1: add + save, quit; run 2: display	Krish Singh ; 1 record(s)	As expected	PASS

Summary: 41/41 end-to-end tests PASS; 84/84 unit checks PASS (no memory errors or leaks); 9/9 compile-fail proofs PASS.

12. Sample Screens / Output

```
T19  Apply for leave (valid)                                          PASS

  [ main menu ]

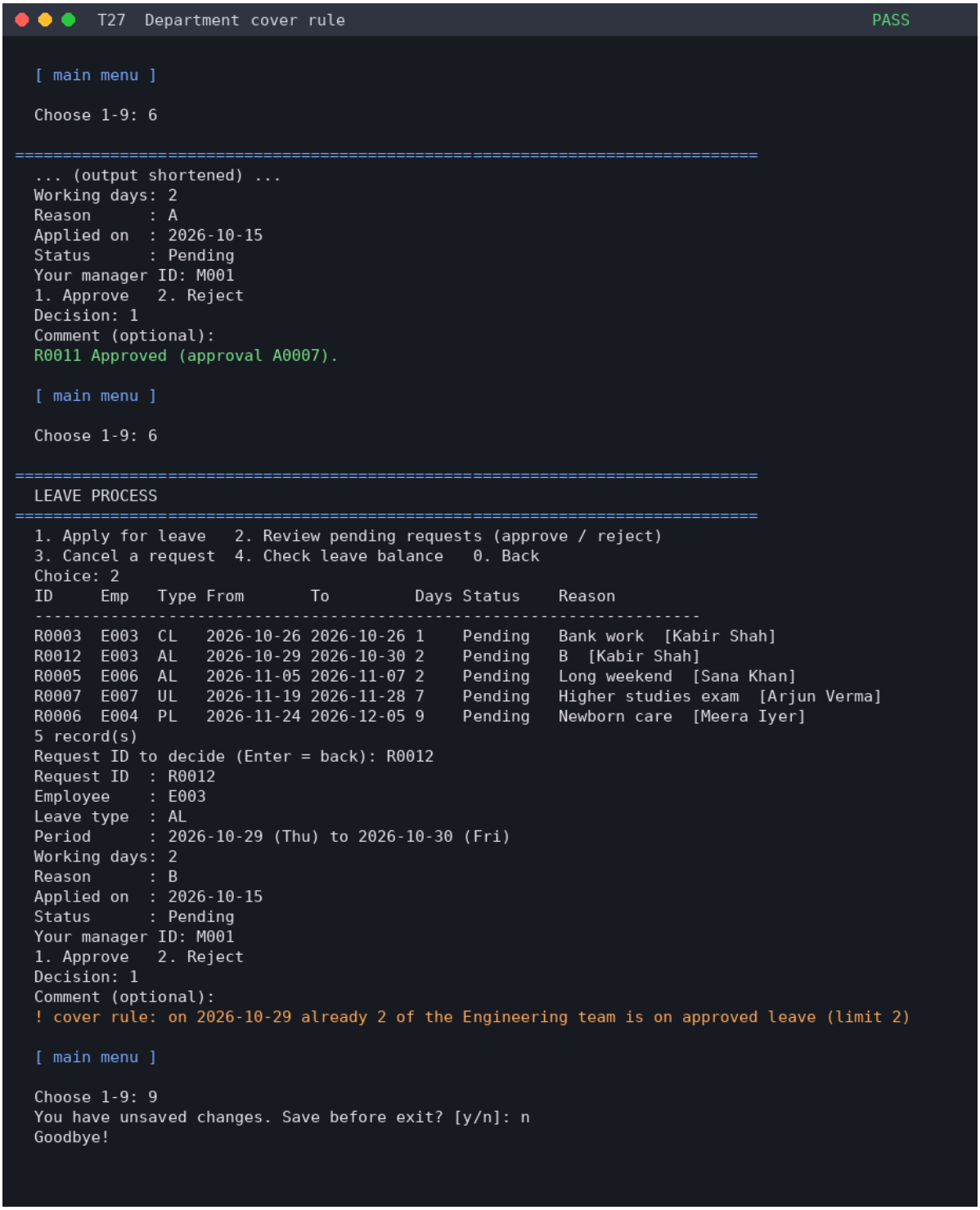
  Choose 1-9: 6

================================================================================
  LEAVE PROCESS
================================================================================
  1. Apply for leave   2. Review pending requests (approve / reject)
  3. Cancel a request  4. Check leave balance    0. Back
  Choice: 1
  Employee ID: E001
  Applying for: Aarav Sharma (Engineering)
  Leave types:
    AL Annual Leave: quota 18/yr, max 15 days/request, notice 3d, back-date 0d, paid
    SL Sick Leave: quota 10/yr, max 7 days/request, notice 0d, back-date 7d, paid
    CL Casual Leave: quota 7/yr, max 3 days/request, notice 0d, back-date 1d, paid
    PL Parental Leave: quota 30/yr, max 30 days/request, notice 14d, back-date 0d, paid
    UL Unpaid Leave: quota unlimited/yr, max 30 days/request, notice 7d, back-date 0d, unpaid
  Type code: AL
  From (YYYY-MM-DD): 2026-11-16
  To (YYYY-MM-DD): 2026-11-17
  Reason: Visit home
  Request R0011 submitted: 2 working day(s), status Pending.
  Balance after pending requests: 13 of 18 day(s).

  [ main menu ]

  Choose 1-9: 9
  You have unsaved changes. Save before exit? [y/n]: n
  Goodbye!
```

Applying for leave (T19)



Department cover rule blocking an approval (T27)

```
T32  Report: monthly calendar                                                PASS

  [ main menu ]

  Choose 1-9: 7

================================================================================
  REPORTS
================================================================================
  1. Employee leave balance   2. Department summary   3. Who is on leave on a date
  4. Monthly leave calendar   5. Overall summary      6. Export all requests to CSV
  7. Audit log (last 15)      0. Back
  Choice: 4
  Year: 2026
  Month (1-12): 10
  Leave calendar - October 2026   (UPPER = approved, lower = pending, _ = weekend, * = holiday)

              1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
              T F S S M T W T F S S M T W T F S S M T W T F S S M T W T F S
  Aarav Sharma. * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ . . . . A A _
  Diya Patel  . * _ _ . . . . S _ _ S . . . . _ _ . . . . . _ _ . . . . . _
  Kabir Shah  . * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ c . . . . _
  Meera Iyer  . * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ . . . . . _
  Vikram Rao  . * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ . . . . . _
  Sana Khan   . * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ . . . . . _
  Arjun Verma . * _ _ . . . . . _ _ . . . . . _ _ . A . . . _ _ . . . . . _
  Neha Gupta  . * _ _ . . . . . _ _ . . . . . _ _ . . . . . _ _ . . . . . _

  [ main menu ]

  Choose 1-9: 9
  Goodbye!
```

Monthly leave calendar (T32)

```
T40  Persistence across restarts (second run)                                PASS

  [ main menu ]

  Choose 1-9: 2

================================================================================
  DISPLAY RECORDS
================================================================================
  1. Employees  2. Managers  3. Leave requests  4. Approvals  5. Holidays
  6. Everything (polymorphic)  7. Leave policies  0. Back
  Choice: 1
  ID    Name               Department    Designation         Joined
  ------------------------------------------------------------------
  E001  Krish Singh        Engineering   Developer           2025-08-01
  1 record(s)

  [ main menu ]

  Choose 1-9: 9
  Goodbye!
```

Data still present after restart (T40)

13. Limitations

- Console interface only; no login/password - the manager enters their own ID when deciding.
- Half-day leave and leave that crosses a year end are not supported (split into two requests).
- Weekends are fixed as Saturday/Sunday; no per-employee shift calendars.
- Text files are used instead of a database, so there is no multi-user concurrency.

14. Future Scope

- Login with roles and password hashing.
- Half-day leave, carry-forward of unused balance, pro-rata quota for new joiners.
- E-mail notification to managers and employees.
- Replace files by SQLite and add a GUI or web front end.

15. Conclusion

The project delivers a complete leave-management workflow with clean separation between data classes, business rules, user interface and storage. It demonstrates every required C++ concept in a working program, goes beyond the blueprint with realistic HR rules, reports, audit logging and robust file handling, and is backed by automated tests and sanitizer runs.